RELATÓRIO TÉCNICO

Aluno: [NOME COMPLETO]
Disciplina: Sistemas Cognitivos com Large Language Models
Título do Projeto: Assistente de Suporte Técnico Baseado em RAG para Documentação Promob

1. O PROBLEMA

Desenvolvi um assistente de perguntas e respostas que responde dúvidas sobre documentação de suporte técnico (Promob), usando apenas a informação presente nos documentos e recusando responder quando a pergunta foge do escopo.

Cheguei a esse escopo depois de tentar, numa etapa anterior, resumir uma transcrição de aula de 2h com o mesmo pipeline RAG. Percebi que a recuperação por similaridade (top-k) não é adequada para cobrir um documento inteiro — ela é eficaz para localizar um trecho específico, não para sumarizar o todo. Por isso migrei para um caso de uso mais compatível com essa arquitetura: QA sobre manuais curtos.

2. CORPUS

Três artigos reais do suporte da empresa onde trabalho (Promob), em Markdown:
- Portal - Renovar Assinaturas
- Promob - Atualizar o Promob
- Promob - Requisitos do computador

São documentos curtos e de tópico bem definido, o que favorece a recuperação por similaridade.

3. POR QUE USAR LLM AQUI

A tarefa não é apenas localizar um trecho, mas interpretar a pergunta, combinar informação de mais de um chunk quando necessário, adaptar o formato da resposta e, principalmente, reconhecer quando a pergunta não tem resposta na base — algo que um buscador por palavra-chave não faz.

4. ARQUITETURA

Documentos .md -> chunking -> embeddings -> ChromaDB -> retriever (top-k) -> prompt com contexto -> LLM local -> resposta.

Stack: LangChain (LCEL), Hugging Face Transformers, ChromaDB. Execução local no Google Colab com GPU T4 (15GB de VRAM).

5. MODELOS TESTADOS

Embeddings: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 — modelo leve, multilíngue, com bom suporte a português, e que deixa VRAM disponível para o LLM de geração.

Para o LLM, testei três opções:

- mT5_multilingual_XLSum (580M) — descartado logo no início. É treinado exclusivamente para sumarização e não segue instruções: ao receber um prompt com instrução explícita, devolveu parte do próprio prompt como resposta.

- LaMini-Flan-T5-248M (248M) — é instruction-tuned, mas apresentou desempenho fraco em português. Nos testes, alternava para inglês no meio da resposta e, em algumas perguntas, recusou-se a responder citando uma suposta "política de uso da OpenAI", que não tem relação nenhuma com este projeto — provavelmente um padrão herdado dos dados de treino do modelo.

- mt0-xl (3.7B) — modelo escolhido para a versão final. Segue instruções de forma consistente e respondeu em português na maioria dos casos. Como contrapartida, exige bem mais VRAM (foi necessário carregá-lo em fp16 para evitar erro de memória — detalhes na seção 7).

Não cheguei a testar LLMs causais como Qwen2.5-Instruct ou Gemma-2-it, pois exigiriam adaptar a task do pipeline para "text-generation" com chat template. Fica registrado como próximo passo.

6. CHUNKING E RETRIEVAL

RecursiveCharacterTextSplitter, chunk_size=500, overlap=50. Como os documentos são curtos, um chunk desse tamanho normalmente cobre um procedimento inteiro.

Retriever: ChromaDB, busca por similaridade, top_k=3. Funcionou bem para este corpus: nas perguntas testadas, a resposta correta praticamente sempre estava concentrada em um único chunk, e o retriever conseguiu localizá-lo.

7. EXECUÇÃO LOCAL

Optei por executar tudo localmente, sem uso de API paga. Vantagens: privacidade (nenhum dado sai do ambiente), custo zero por chamada e controle total sobre o modelo. Desvantagem: latência maior do que uma API cloud otimizada.

Enfrentei um problema real nessa etapa: o mt0-xl (3.7B) gerou um erro de memória (OutOfMemoryError) na primeira tentativa. A causa foi o pipeline carregando o modelo em fp32 por padrão, consumindo cerca de 14.8GB, quase o limite da T4. A correção foi forçar torch_dtype=torch.float16, o que reduziu o consumo de VRAM praticamente à metade.

8. PROMPT ENGINEERING — AS 3 TÉCNICAS

==========
Testei zero-shot, few-shot e structured output (JSON), todas com as mesmas 5 perguntas e o mesmo modelo (mt0-xl), para manter a comparação controlada.

Zero-shot: instrução direta, com regra explícita de recusa ("se não estiver no contexto, responda exatamente 'Não localizei essa informação no documento'").

Few-shot: mesma lógica de grounding, acrescida de um exemplo de pergunta e resposta antes da pergunta real.

Structured output: solicita a resposta em formato JSON: {"resposta": "...", "encontrado_no_documento": true/false}.

Perguntas utilizadas (uma por documento, mais uma fora do escopo, para testar se o modelo aluciona):
1. Como faço para renovar minha assinatura do Promob?
2. Quantos GB de RAM são recomendados para rodar o Promob?
3. É possível atualizar o Promob com o programa fechado?
4. O Promob funciona em máquina virtual?
5. Qual o preço da licença anual do Promob? (fora do escopo — não consta em nenhum documento)

9. RESULTADOS
==========

ZERO-SHOT (mt0-xl):
1. Correta — reproduziu os passos corretos do documento de renovação.
2. Incorreta — respondeu "4 GB - 8 GB", que corresponde ao espaço em disco, não à RAM recomendada (16GB). Houve confusão entre duas linhas da mesma tabela.
3. Correta no conteúdo ("Yes"), mas em inglês em vez de português.
4. Correta ("não").
5. Correta — recusou responder, sem inventar informação.

FEW-SHOT (mt0-xl):
1. Incorreta — respondeu com o procedimento de ATUALIZAÇÃO, não de renovação. O modelo parece ter reproduzido o conteúdo do exemplo usado no prompt (que também tratava de atualização) em vez de responder à pergunta real.
2. Mesma confusão entre RAM e espaço em disco.
3 e 4. Corretas no conteúdo, novamente em inglês.
5. Alucinou — respondeu "Gratuita", informação inexistente na base.

STRUCTURED OUTPUT (mt0-xl):
Nenhuma das 5 respostas veio em formato JSON válido; o modelo não seguiu o formato solicitado em nenhum caso. Na pergunta 5 (fora do escopo), alucinou o valor "$10,00".

LAMINI (248M), para efeito de comparação de modelo:
Testei zero-shot e few-shot também neste modelo. Em ambos os casos o resultado foi insatisfatório: alternância para inglês no meio das

respostas, reprodução de trechos do próprio prompt como se fossem resposta, e recusas infundadas citando políticas que não existem neste projeto. A troca de técnica de prompt não alterou esse padrão, o que sugere que a limitação está no tamanho/capacidade do modelo, não na técnica utilizada.

10. PRINCIPAIS CONCLUSÕES

- Das 3 técnicas, apenas o zero-shot recusou corretamente a pergunta fora de escopo; as outras duas alucinaram. A diferença não está na técnica em si, mas no fato de que só o template zero-shot continha a instrução de recusa explícita ("responda exatamente..."). A lição principal é que essa instrução precisa estar presente em qualquer prompt, independentemente da técnica escolhida.

- O few-shot trouxe um risco que eu não esperava: se o exemplo usado no prompt for temáticamente próximo de outras perguntas do domínio, o modelo pode reproduzir o conteúdo do exemplo em vez de aplicar apenas seu estilo. Foi o que ocorreu na pergunta 1.

- Nenhum dos modelos testados seguiu o formato JSON de maneira confiável, mesmo sendo instruction-tuned. Seguir o conteúdo de uma instrução e seguir um formato rígido parecem ser capacidades distintas.

- O modelo maior (mt0-xl, 3.7B) superou claramente o menor (LaMini, 248M) em qualidade e consistência no idioma, o que justificou o custo adicional de VRAM.

- Identifiquei uma falha estrutural: a pergunta sobre RAM sempre retornava o valor errado (espaço em disco). Provavelmente o chunking por caractere não preserva bem a estrutura de uma tabela Markdown, misturando as duas informações no mesmo chunk.

Não testei o cenário de ausência total de contexto recuperado (LLM "puro"). Usei a pergunta fora de escopo como aproximação, já que o retriever ainda retorna chunks irrelevantes para ela.

11. RISCOS E CONTROLES

- Alucinação: mitigada pela instrução de recusa explícita, mas essa instrução só estava presente no template zero-shot. Próximo passo natural é replicá-la em todos os templates.
- Prompt injection via documento: risco baixo neste projeto, já que a base é controlada internamente, mas seria um cuidado necessário caso a base incorporasse conteúdo externo no futuro.
- Dados sensíveis: não é um problema nesta versão, pois os documentos já são públicos, mas a execução local já mitigaria esse risco caso a base viesse a incluir dados sensíveis.

12. COMO REPRODUZIR

1. Colab com GPU (T4 ou superior).
2. Colocar os arquivos .md na pasta /content/docs.
3. Executar a célula de instalação de dependências.
4. Ajustar parâmetros (modelo, chunk_size, top_k, estratégia de prompt), se necessário.
5. Executar main().
6. Ao trocar de configuração sem reiniciar o kernel, executar chromadb.api.client.SharedSystemClient.clear_system_cache() após apagar a pasta do vector store antigo, para evitar erro de banco somente-leitura.

13. LIMITAÇÕES

- Chunking genérico não lida bem com tabelas Markdown (causou o erro de RAM x espaço em disco).
- Nenhum modelo sustentou saída em JSON de forma confiável.
- Avaliação das respostas foi manual, sem métrica automática.
- Não testei isoladamente o cenário sem nenhum contexto recuperado.

14. PRÓXIMOS PASSOS

- Padronizar a instrução de recusa explícita em todos os templates, não apenas no zero-shot.
- Substituir o exemplo do few-shot por um de tema distinto dos documentos reais, para reduzir o risco de contaminação.
- Testar uma abordagem melhor para processar tabelas antes do chunking.
- Testar modelos causais mais recentes (Qwen2.5-3B-Instruct, Gemma-2-2b-it).
- Reforçar a instrução de JSON com um exemplo de saída válida no próprio prompt.